Your Name: Caleb Zhong

Your Andrew ID: yuzhong4

Homework 4

0 Introduction

0.1 Collaboration and Originality

1. Did you receive help of any kind from anyone (other than the instructor or TAs) in developing your software for this assignment (Yes or No)? If you answered Yes, provide the name(s) of anyone who provided help, and describe the type of help that you received. No

2. Did you give help of any kind to anyone in developing their software for this assignment (Yes or No)? If you answered Yes, provide the name(s) of anyone that you helped, and describe the type of help that you provided. No

3. Did you examine anyone else's software for this assignment (Yes or No)? Do not describe software provided by the instructor. No

4. Are you (or the course instructor) the author of every line of source code submitted for this assignment (Yes or No)? If you answered No: Yes
    a. identify the software that you did not write,
    b. explain where it came from, and
    c. explain why you used it.

5. Are you the author of every word of your report (Yes or No)? If you answered No: Yes
    a. identify the text that you did not write,
    b. explain where it came from, and
    c. explain why you used it.

0.2 Instructions

Some experiments require you to set parameters or weights. You must explain why you chose particular values, and how your choices relate to how the technique works. We look for good experimental design – parameters that explore interesting issues or hypotheses (even if the hypothesis turns out to be wrong).

You must analyze the experimental results. Don't just summarize the numbers contained in the table – we can read the table ourselves. Instead, explain what conclusions you can reach based on the experiment. We are primarily interested in your observations about general trends, not quirky queries. Usually a good analysis addresses several issues. Show that you understand what the results mean, based upon what we have discussed in class.

Leave the page breaks between sections, as shown in this file. This is will reduce the number of duplicate pages in your graded report.

Instructions are shown in a red italic bold font. <u>Do not</u> include instructions in your report. For example, <u>delete this page</u>. There is a <u>2 point deduction</u> for leaving instructions in your report (because it creates extra work for the TAs).

1 Experiment: Effect of Reranking Depth

1.1 Parameters for Reranking Depth Experiment

For the reranking depth experiment, BM25 was used to generate the initial ranking of 1,000 documents for each query with $k_1 = 1.2$ and $b = 0.75$.

The same HW3 training data were reused for the LTR models: HW3-train.qry and HW3-train.qrel.

Based on the HW3 feature-combination experiment, the LTR configuration used only features f4, f5, f8, and f17, with all other features disabled.

Coordinate Ascent was implemented with RankLib model 4, and ListNet was implemented with RankLib model 7. The LTR feature computation used QL mu = 1500.

The reranking depths tested were 100, 250, and 500.

For BERT reranking, the two provided cross-encoder models were used: `ms-marco-MiniLM-L-6-v2` and `ms-marco-MiniLM-L-12-v2`.

Both BERT models used the same passage settings required by the experiment: `psgLen=150, psgStride=125, psgCnt=3, scoreAggregation=maxp, and maxTitleLength=16.`

1.2 Experimental Results for Reranking Depth

	BM25 (Exp-1.1a)	Coordinate Ascent Reranking Depth		
		100 (Exp-1.1b)	250 (Exp-1.1c)	500 (Exp-1.1d)
MRR@k	0.3790	0.5809	0.5809	0.5809
P@10	0.3083	0.4250	0.4250	0.4250
P@20	0.3083	0.3625	0.3625	0.3625
P@30	0.2889	0.3500	0.3500	0.350
MAP@1K	0.2809	0.3358	0.3348	0.3345
NDCG@10	0.2557	0.3659	0.3659	0.3659
NDCG@20	0.2695	0.3623	0.3623	0.3623
NDCG@30	0.2867	0.3874	0.3874	0.3874
R@100	0.7855	0.7855	0.8014	0.8014
R@500	0.9734	0.9734	0.9734	0.9734
R@1000	0.9914	0.9914	0.9914	0.9914
Run Time	00:22	00:19	00:19	00:21

	BM25 (n/a)	ListNet Reranking Depth		
		100 (Exp-1.2b)	250 (Exp-1.2c)	500 (Exp-1.2d)
MRR@k	0.3790	0.7384	0.7318	0.7318
P@10	0.3083	0.4333	0.4250	0.4250
P@20	0.3083	0.3208	0.3167	0.3167
P@30	0.2889	0.2944	0.2861	0.2861
MAP@1K	0.2809	0.3771	0.3642	0.3592
NDCG@10	0.2557	0.4203	0.4171	0.4171
NDCG@20	0.2695	0.3883	0.3853	0.3853
NDCG@30	0.2867	0.4036	0.3947	0.3947
R@100	0.7855	0.7855	0.6643	0.6591
R@500	0.9734	0.9734	0.9734	0.9734
R@1000	0.9914	0.9914	0.9914	0.9914
Run Time	00:22	00:16	00:17	00:20

	BM25 (n/a)	Coordinate Ascent Reranking Depth	BERT, 6 layers Reranking Depth		
		500 (n/a)	100 (Exp-1.3b)	250 (Exp-1.3c)	500 (Exp-1.3d)
MRR@k	0.3790	0.5809	0.6871	0.6802	0.6802
P@10	0.3083	0.4250	0.4667	0.4417	0.4417
P@20	0.3083	0.3625	0.4208	0.4208	0.4208
P@30	0.2889	0.350	0.3778	0.3750	0.3694
MAP@1K	0.2809	0.3345	0.3790	0.3727	0.3682
NDCG@10	0.2557	0.3659	0.3905	0.3688	0.3688
NDCG@20	0.2695	0.3623	0.4301	0.4209	0.4206
NDCG@30	0.2867	0.3874	0.4493	0.4408	0.4368
R@100	0.7855	0.8014	0.7855	0.7423	0.7400
R@500	0.9734	0.9734	0.9734	0.9734	0.9734
R@1000	0.9914	0.9914	0.9914	0.9914	0.9914
Run Time	00:22	00:21	01:14	03:16	06:47

	BM25 (n/a)	Coordinate Ascent Reranking Depth	BERT, 12 layers Reranking Depth		
		500 (n/a)	100 (Exp-1.4b)	250 (Exp-1.4c)	500 (Exp-1.4d)
MRR@k	0.3790	0.5809	0.6696	0.6446	0.6446
P@10	0.3083	0.4250	0.4833	0.4667	0.4667
P@20	0.3083	0.3625	0.4542	0.4333	0.4333
P@30	0.2889	0.350	0.4139	0.4056	0.4000
MAP@1K	0.2809	0.3345	0.3928	0.3839	0.3786
NDCG@10	0.2557	0.3659	0.3937	0.3800	0.3800
NDCG@20	0.2695	0.3623	0.4413	0.4152	0.4151
NDCG@30	0.2867	0.3874	0.4814	0.4667	0.4628
R@100	0.7855	0.8014	0.7855	0.7827	0.7518
R@500	0.9734	0.9734	0.9734	0.9734	0.9734
R@1000	0.9914	0.9914	0.9914	0.9914	0.9914
Run Time	00:22	00:21	02:38	06:32	12:36

1.3 Discussion of Reranking Depth Experimental Results

From the results, it’s clear that the BERT rerankers consistently outperform both the BM25 baseline and the traditional LTR methods in terms of top-tier precision. While BM25 relies on lexical term matching, LTR methods rely on manually picked features that often just count word overlaps, the BERT rerankers, by contrast, leverage deep contextual embeddings to bridge the semantic gap between queries and documents, allowing them to identify relevance that is invisible to the statistical heuristics of BM25 or even the manually engineered features of Coordinate Ascent and ListNet. This is why even with a shallow reranking depth of 100, the BERT models achieve much higher precision and MRR than any of the traditional baselines. It basically proves that semantic matching is way more effective than just calculating statistical heuristics.

Building upon this initial observation, an unexpected trend emerges when analyzing the impact of reranking depth on BERT’s performance. Conventionally, one might expect that allowing a sophisticated model like BERT to examine more documents (increasing depth from 100 to 500) would yield higher precision as it can find more relevant stuff. However, the data shows the opposite that for both the 6-layer and 12-layer models, the MRR and P@10 actually peak at a depth of 100 and then drop as the depth increases. I think this happens because the lower-quality documents from the tail of the BM25 list act as "semantic noise." Since BERT is so sensitive to context, it might get "distracted" by a document that sounds similar to the query topic but isn't actually a good match, ranking it too high and pushing down the real answers.

Comparing the two BERT models shows a bit of a trade-off between power and stability. The 12-layer model is definitely the "smartest" at a depth of 100, hitting the highest NDCG scores in the whole experiment. But it’s also the most sensitive to that noise I mentioned earlier cause its MRR took a much bigger hit (dropping from 0.6696 to 0.6446) when the depth increased to 500. Its complexity makes it more prone to over-interpreting the false-positive signals found in a noisier, larger document pool. In contrast, the 6-layer model seems a bit more robust, it’s not quite as precise at the very top, but it doesn't get as confused by the lower-quality documents at higher depths.

Finally, the run-time data really puts things into perspective. Reranking 500 documents with the 12-layer BERT took over 12 minutes, which is incredibly slow compared to the 21 seconds it took for Coordinate Ascent. Since reranking 500 docs was not only slower but also gave worse precision than reranking just 100, it’s a pretty bad trade-off. This suggests that for practical search engine design, the "sweet spot" for BERT is as a high-precision, final-stage reranker applied only to a small, high-quality set of candidates.

2 Experiment: Passages and Aggregation Methods

2.1 Parameters for Passages and Aggregation Methods

For the passages and aggregation methods experiment, BM25 was used to generate the initial ranking of 1,000 documents for each query with $k_1 = 1.2$ and $b = 0.75$.

The top 250 documents were reranked by BERT for every run.

The experiment used the HW3 bag-of-words query set `HW3-Exp-Bow.qry`.

To isolate the effect of passage construction and score aggregation, the BERT model and title setting were held fixed across all runs. The model used was `ms-marco-MiniLM-L-6-v2`, with `maxTitleLength=16`.

Six passage configurations were tested:

```
(psgLen, psgStride, psgCnt) = (50,25,3), (50,25,6), (100,75,3), (100,75,6),
(200,175,3), (200,175,6).
```

Three score aggregation methods were evaluated: **firstp**, **maxp**, and **avgp**.

For firstp, only the 3-passage variants were reported because the document score depends only on the first passage, so changing `psgCnt` from 3 to 6 does not affect the result.

2.2 Experimental Results for Passages and Aggregation Methods

	FirstP PsgLen / PsgStride / PsgCnt					
	50// (Exp-2.1a)	**(n/a)**	**100// (Exp-2.1c)**	**(n/a)**	**200// (Exp-2.1e)**	**(n/a)**
MRR@k	0.5222		0.5812		0.5911	
P@10 **P@20** **P@30**	0.3667 0.3708 0.3194		0.4083 0.3708 0.3306		0.4500 0.3667 0.3333	
MAP@1K	0.3365		0.3445		0.3428	
NDCG@10 **NDCG@20** **NDCG@30**	0.3425 0.3886 0.3965		0.3887 0.3963 0.4170		0.3944 0.3931 0.4145	
R@100 **R@500** **R@1000**	0.6485 0.9734 0.9914		0.6506 0.9734 0.9914		0.6700 0.9734 0.9914	
Run Time	01:21		02:18		04:24	

	MaxP PsgLen / PsgStride / PsgCnt					
	50/25/3 (Exp-2.2a)	**50/25/6 (Exp-2.2b)**	**100/75/3 (Exp-2.2c)**	**100/75/6 (Exp-2.2d)**	**200/175/3 (Exp-2.2e)**	**200/175/6 (Exp-2.2f)**
MRR@k	0.5764	0.6417	0.6833	0.7118	0.6389	0.6406
P@10 **P@20** **P@30**	0.4167 0.3542 0.3167	0.4417 0.3958 0.3639	0.4417 0.4000 0.3472	0.4667 0.3958 0.3778	0.3833 0.3958 0.3500	0.4083 0.4125 0.3667
MAP@1K	0.3404	0.3642	0.3587	0.3972	0.3636	0.3846
NDCG@10 **NDCG@20** **NDCG@30**	0.3984 0.3882 0.4021	0.4002 0.4191 0.4418	0.3930 0.4095 0.4230	0.4132 0.4188 0.4644	0.3403 0.4015 0.4157	0.3533 0.4055 0.4361
R@100 **R@500** **R@1000**	0.6458 0.9734 0.9914	0.7011 0.9734 0.9914	0.7356 0.9734 0.9914	0.766 0.9734 0.9914	0.7584 0.9734 0.9914	0.8166 0.9734 0.9914
Run Time	01:39	02:54	02:28	04:28	04:26	06:58

	AvgP PsgLen / PsgStride / PsgCnt					
	50/25/3 (Exp-2.3a)	**50/25/6 (Exp-2.3b)**	**100/75/3 (Exp-2.3c)**	**100/75/6 (Exp-2.3d)**	**200/175/3 (Exp-2.3e)**	**200/175/6 (Exp-2.3f)**
MRR@k	0.6349	0.6694	0.6167	0.5675	0.6577	0.6175
P@10	0.4417	0.4750	0.4167	0.3917	0.3917	0.3917
P@20	0.3667	0.3750	0.3875	0.3625	0.3458	0.3292
P@30	0.3222	0.3167	0.3250	0.3250	0.3056	0.3028
MAP@1K	0.3793	0.3852	0.3524	0.3421	0.3382	0.3352
NDCG@10	0.4329	0.4251	0.3786	0.3570	0.3665	0.3617
NDCG@20	0.4332	0.4155	0.3993	0.3623	0.3498	0.3508
NDCG@30	0.4420	0.4139	0.3982	0.3956	0.3716	0.3777
R@100	0.6521	0.6714	0.6913	0.7358	0.7397	0.7607
R@500	0.9734	0.9734	0.9734	0.9734	0.9734	0.9734
R@1000	0.9914	0.9914	0.9914	0.9914	0.9914	0.9914
Run Time	01:35	02:59	02:32	04:31	04:54	08:01

2.3 Discussion for Passages and Aggregation Methods

The experimental results for passage formation and score aggregation reveal that how we slice the documents and combine the scores makes a huge difference for BERT. Among the three aggregation strategies, MaxP emerges as the superior method, achieving the highest overall performance with an MRR of 0.7118 and a MAP@1K of 0.3972 in the 100/75/6 configuration. It suggests that in most search tasks, relevance is often concentrated in a specific passage rather than being distributed uniformly. MaxP acts like a "needle-in-a-haystack" tool, it just needs to find that one perfect passage to give the whole document a high score, even if the rest of the text is irrelevant.

In contrast, the AvgP strategy seems to struggle as the number of passages and the passage length increase. While AvgP performs reasonably well with very short, focused passages (50/25/6 reaching an MRR of 0.6694), its performance drops to an MRR of 0.5675 when moving to the 100/75/6 setup. This decline occurs because AvgP averages everything. As the search engine captures more passages of text, it inevitably includes more non-relevant background information. However, AvgP treats this extra noise as a signal of lower overall document quality, thereby dragging down the scores of passages that might actually contain a perfect and relevant answer. Furthermore, FirstP shows that the beginning of a document is usually a strong signal (reaching 0.5911 MRR), but it just can't compete with the broader coverage provided by MaxP while looking at more of the document.

The choice of passage length and stride illustrates a balance between semantic context and signal noise. For MaxP, the 100-token length appears to be the "sweet spot". Passages of only 50 tokens likely lack the necessary context for BERT to resolve semantic dependencies, leading to lower precision (MRR 0.5764 for 50/25/3). But when it extends the length to 200 tokens, the performance didn't really get better and sometimes even got worse, as seen in the drop in P@10 from 0.4667 (100/75/6) to 0.4083 (200/175/6). This is likely because longer passages are more likely to be truncated by BERT's internal 512-WordPiece limit or simply contain too much off-topic text.

Finally, we can see the contrast in how the number of passages (PsgCnt) affected the models. For MaxP, increasing the count from 3 to 6 passages consistently improves performance across all configurations, such as the jump from 0.6833 to 0.7118 in the 100/75 setup, because it increases the probability of hitting a highly relevant passage. For AvgP, however, more passages actually make things worse, with the MRR falling from 0.6167 to 0.5675 in the 100/75 config. This further confirms that while broader document coverage is beneficial for identifying specific relevant evidence, it is actively harmful when using aggregation methods that do not distinguish between the "good" parts and the "noise". Therefore, the optimal configuration for a BERT reranker seems to use a moderate passage length (100 tokens) with a high passage count and MaxP aggregation method.

3 Experiment: Reranking Configurations

3.1 Descriptions of Reranking Configurations

In this experiment, I varies three aspects of the pipeline design: (1) the initial candidate generator, using either BM25 or RankedBoolean; (2) the type of intermediate reranker, using either learning-to-rank (LTR) or BERT; (3) whether query expansion (PRF) remains beneficial in the presence of strong neural rerankers; and (4) the final stage, which in most cases was BERT, but in one configuration was LTR in order to test whether an explicit feature-based correction stage could improve the final ranking.

Configuration 1: BM25 → SVMrank → BERT-12L
The first configuration was designed as an accuracy-first pipeline:

No PRF

Stage 1: **BM25 (1000)**

- $k_1 = 1.2, b = 0.75$

Stage 2: **SVMrank (250)**

- `C=0.001`
- features = {f4,f5,f8,f17}

Stage 3: **BERT-12L (100)**

- `psgLen=100, psgStride=75, psgCnt=6`
- scoreAggregation = maxp
- `maxTitleLength=16`

The intuition behind this design was to combine robust feature-based filtering, and the strongest semantic reranker at the end. BM25 is used to generate a high-recall lexical candidate set, followed by SVMrank with the compact "Core Four" feature (Proved to have best performance in HW3) set to remove obvious low-quality or weakly matched documents while preserving a reasonably candidate depth. The final stage used the 12-layer MiniLM BERT reranker on the top 100 documents, because earlier results suggested that the deeper model is strongest when applied to a relatively clean and shallow set. I expected this configuration to serve as a strong reference point for comparing the effect of PRF, lighter rerankers, and alternative initial rankers.

Configuration 2: RM3 PRF → BM25 → SVMrank → BERT-12L
The second configuration was designed to test whether PRF remains beneficial:

Query rewrite: **RM3 PRF**

- `numDocs=10, numTerms=10`

- expansion input = `keywords`, expansion output = `body`

Stage 1: **BM25 (1000)**

- $k_1 = 1.2, b = 0.75$

Stage 2: **SVMrank (250)**

- `C=0.001`
- features = {f4,f5,f8,f17}

Stage 3: **BERT-12L (100)**

- `psgLen=100, psgStride=75, psgCnt=6`
- scoreAggregation = maxp
- `maxTitleLength=16`

This configuration is identical to Configuration 1 except that it introduces RM3-based query expansion. It is included specifically as a controlled experiment to evaluate the impact of PRF in a modern multi-stage pipeline with BERT.

I selected RM3 with keyword-based feedback because earlier experiments suggested that cleaner fields may produce more focused expansion terms than noisy body text. I also chose a low origWeight so that the expanded query would make full use of the feedback evidence. The hypothesis was that better query formulation at the start of the pipeline might increase the quality of the BM25 candidate set and consequently benefit all later reranking stages.

Configuration 3: BM25 → ListNet → BERT-6L
The third configuration was designed as a more balanced and practical pipeline:

No PRF

Stage 1: **BM25 (1000)**

- $k_1 = 1.2, b = 0.75$

Stage 2: **ListNet(100)**

- features = {f4,f5,f8,f17}

Stage 3: **BERT-6L(100)**

- `psgLen=100, psgStride=75, psgCnt=6`
- scoreAggregation = maxp

- `maxTitleLength=16`

This configuration intentionally avoided PRF and the heavier 12-layer BERT model. The goal was to test whether a simpler and more efficient pipeline could still obtain strong performance by combining a lightweight LTR stage with a relatively cheap semantic reranker. I used BM25 to generate the initial candidate pool, then ListNet with the same compact feature set to provide a first round of reranking. Finally, I applied the 6-layer BERT reranker to the same top 100 documents. The hypothesis was that this design would provide a better balance between effectiveness and efficiency, preserving much of the benefit of semantic reranking while reducing overall complexity and runtime.

Configuration 4: RankedBoolean → SVMrank → BERT-12L
The fourth configuration was designed to test RankedBoolean as a high-variance initial ranker:

No PRF

Stage 1: **RankedBoolean (1000)**

Stage 2: **SVMrank (250)**

- `C=0.001`
- features = {f4,f5,f8,f17}

Stage 3: **BERT-12L (100)**

- `psgLen=100, psgStride=75, psgCnt=6`
- scoreAggregation = maxp
- `maxTitleLength=16`

This configuration was trying to explore a meaningful hypothesis about RankedBoolean role in a multi-stage search engine. Compared with BM25, RankedBoolean may retrieve a candidate set that contains more documents satisfying hard lexical constraints but with less reliable ranking quality. I therefore treated it as a deliberately noisier initial stage. To compensate for this, I used a strong two-step cleanup process: first SVMrank with the compact feature set, then the stronger 12-layer BERT reranker for final precision. The hypothesis was that RankedBoolean might increase recall for certain strictly lexical queries, while the later stages would repair its weaker ranking quality.

Configuration 5: BM25 → BERT-6L → BERT-12L
The fifth configuration was a two-BERT cascade:

No PRF

Stage 1: **BM25 (1000)**

- $k_1 = 1.2, b = 0.75$

Stage 2: **BERT-6L(250)**

- `psgLen=50, psgStride=25, psgCnt=6`
- scoreAggregation = avgp
- `maxTitleLength=16`

Stage 3: **BERT-12L(100)**

- `psgLen=100, psgStride=75, psgCnt=6`
- scoreAggregation = maxp
- `maxTitleLength=16`

This configuration was designed specifically to test whether two semantic reranking stages can play different roles in the pipeline. Instead of using the first BERT stage as a final reranker, I used the lighter 6-layer model as an intermediate semantic filter over a broader set of documents. The idea was that a relatively cheap BERT model could remove semantically irrelevant documents before the stronger 12-layer model performs the final ranking. Conceptually, this configuration separates broad semantic filtering from final high-precision semantic ordering. The hypothesis was that the first BERT stage would reduce the noise passed to the second BERT stage, allowing the final model to focus on a cleaner set of candidates. Even if this hypothesis does not hold in practice, it directly tests whether semantic reranking can itself be staged in a coarse-to-fine manner.

Configuration 6: BM25 → BERT-6L → SVMrank
The sixth configuration ended with LTR instead of BERT:

No PRF

Stage 1: **BM25 (1000)**

- $k_1 = 1.2, b = 0.75$

Stage 2: **BERT-6L (250)**

- `psgLen=100, psgStride=75, psgCnt=3`
- scoreAggregation = maxp
- `maxTitleLength=16`

Stage 3: **SVMrank (100)**

- `C=0.001`

- features = {f4,f5,f8,f17}

This configuration was designed to test whether the final stage in a ranking pipeline must always be semantic, or whether it can instead be an explicit feature-based correction layer. BM25 first generates a lexical candidate set. Then I applied BERT-6L on a relatively broad set of documents so that semantic similarity could influence which documents survived to the final stage. However, instead of ending with BERT, I used SVMrank as the final reranker. The intuition was that BERT may occasionally promote documents that are semantically similar but weak in authority, title match, or topical focus. A final LTR stage using PageRank, lexical match, and query term density could correct some of these semantic false positives. My hypothesis was that, for certain query types, explicit quality and focus signals might still be useful even after semantic reranking.

3.2 Experimental Results for Reranking Configurations

	Config$_1$ (Exp-3.1a)	**Config$_2$ (Exp-3.1b)**	**Config$_3$ (Exp-3.1c)**	**Config$_4$ (Exp-3.1d)**	**Config$_5$ (Exp-3.1e)**	**Config$_6$ (Exp-3.1f)**
MRR@k	0.7361	0.3138	0.7257	0.7361	0.7667	0.7500
P@10	0.5167	0.2500	0.5083	0.5250	0.5000	0.4417
P@20	0.4167	0.2958	0.4083	0.4458	0.4292	0.3583
P@30	0.3639	0.2722	0.3833	0.4056	0.3833	0.3139
MAP@1K	0.3892	0.2197	0.4085	0.3997	0.3882	0.3910
NDCG@10	0.4560	0.1704	0.4522	0.4561	0.4388	0.4384
NDCG@20	0.4402	0.2045	0.4449	0.4611	0.4318	0.4169
NDCG@30	0.4616	0.2291	0.4837	0.4954	0.4598	0.4210
R@100	0.7622	0.6099	0.7855	0.7785	0.6714	0.7338
R@500	0.9734	0.8046	0.9734	0.9483	0.9734	0.9734
R@1000	0.9914	0.8270	0.9914	0.9662	0.9914	0.9914
Run Time	03:28	09:32	02:17	04:01	06:35	02:48

3.3 Discussion for Reranking Configurations

Configuration 1 was intended to be our "accuracy-first" standard, and from the data we can see that it definitely lived up to that role with a strong MRR of 0.7361 and a P@10 of 0.5167. My hypothesis for this design was that SVMrank would act as a reliable filter to prune the 1000-document BM25 pool down to a high-quality set of 250, which would then allow the 12-layer BERT to resolve semantic dependencies without being distracted by low-quality passage.

However, when we compare Configuration 1 to Configuration 3, which was my "budget-friendly" version designed for efficiency, we will find that it actually out of expectation. In Config 3, I swapped out SVMrank for ListNet and used the lighter 6-layer BERT instead of the 12-layer one. I expected a noticeable drop in performance, but the data shows it actually held its own remarkably well, hitting an MRR of 0.7257, which is only a tiny bit lower than Config 1. The P@10 was also super close (0.5083 vs. 0.5167). This suggests that for a relatively "clean" pool of 100 documents, the 12-layer BERT might actually be overkill. It seems like the 6-layer model is smart enough to handle most of the semantic heavy lifting, and the extra complexity of the 12-layer model hits a point of diminishing returns. And for practical perspective, Config 3 finished the entire task in just 02:17, while Configuration 1 took 03:28. That means I was able to get about 98% of the precision while being over a minute faster per run. This really supports the hypothesis that a lightweight LTR stage combined with a smaller BERT model is a much more balanced and realistic choice for a search engine.

The most striking observation is the massive failure of Configuration 2, which incorporated RM3-based PRF. While the hypothesis was that query expansion would improve the initial candidate set for the BERT reranker, the data shows an MRR of only 0.3138, which is less than half of most other configurations. This likely happened because the expansion terms introduced too much noise or moved the query intent away from the relevant documents. Even with the highest performed LTR features and a 12-layer BERT stage at the end, the system couldn't recover from a poor initial lexical pool. This proves that while BERT is a powerful reranker, it is still fundamentally limited by the quality of the documents passed to it by the earlier stages of the pipeline.

In contrast, the success of Configuration 5, which used double neural rerankers, confirms that stacking semantic models can be highly effective. This configuration achieved the highest MRR at 0.7667. My hypothesis was that the 6-layer model would act as a coarse filter to remove semantically irrelevant documents before the 12-layer model did the final ordering, and the results back this up. By using two stages of BERT, the pipeline maintains a consistent focus on contextual relevance throughout the reranking process. However, this high performance comes at a cost that it had the second-longest run time, which is nearly three times slower than the more efficient Configuration 3.

For Configuration 4, which used RankedBoolean as its starting point. I expected this to be a "noisy" initial stage, but it actually tied for the highest P@10 at 0.5250 and achieved the best NDCG@30 of 0.4954. This suggests that as long as the initial ranker captures the relevant documents somewhere in the top 1000, the combination of SVMrank and 12-layer BERT is strong enough to "repair" the lack of initial ordering.

Finally, looking at Configuration 6, which ended with SVMrank instead of BERT, gives us reliability that the traditional features can also play as a final stage. While its MRR (0.7500) was very strong, its

precision metrics like P@10 (0.4417) were actually lower than most other pipelines that ended with BERT. This supports the idea that while LTR features like PageRank or term density can help correct some semantic false positives, they aren't quite as effective at the very top of the ranking as the deep linguistic understanding of a BERT model.